

# RETAIL SALES & INVENTORY ANALYTICS PLATFORM

---

## Step-by-Step Project Implementation Guide

**Project scope:** The project is an end-to-end retail data pipeline using **ADF + ADLS + Databricks + Power BI**. Data is brought from multiple sources into the data lake, cleaned in Silver, curated in Gold, and consumed by Power BI.

## Project Architecture

Source Systems → Azure Data Factory → ADLS Gen2 → Databricks → Bronze / Silver / Gold → Power BI

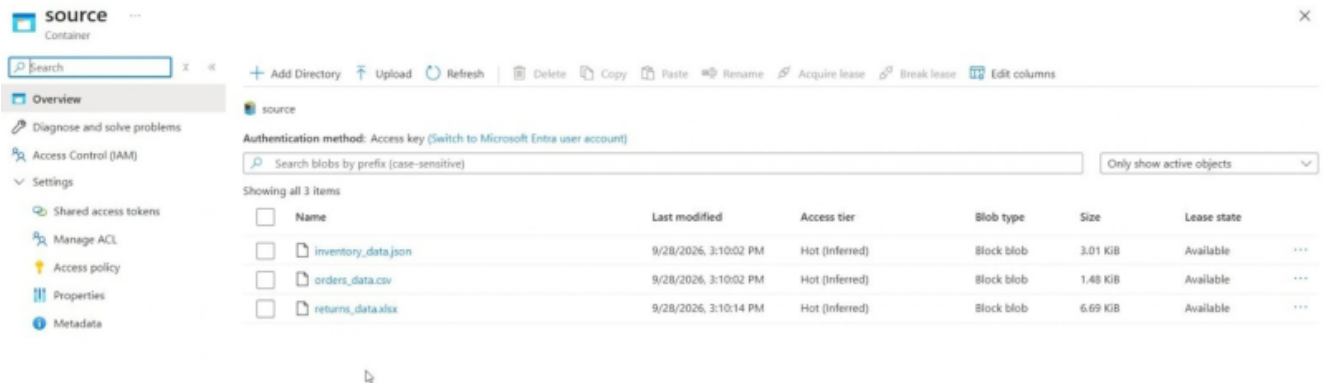
The project uses multiple source systems, a data lake, layered processing, and Power BI reporting.

## Guide structure

Each following page contains one screenshot followed by a separate step-by-step explanation based on the project document and the screenshot.

**ADF + ADLS Gen2 + Databricks + Power BI**

## Step 1: Verify the Source Data in ADLS



ADLS source container showing the incoming retail datasets

### Step-by-step explanation

- Open the **source** container in the storage environment and verify that the incoming retail files are available.
- The screenshot shows **inventory\_data.json**, **orders\_data.csv**, and **returns\_data.xlsx** as source files.
- These files represent the raw inputs that will be brought into the data lake and processed through the retail data pipeline.
- The project document defines a multi-source requirement: transaction/store/product data and customer data are brought into the data lake for downstream processing.

## Step 2: Run the ADF Retail Ingestion Pipeline

The screenshot displays the Microsoft Fabric Data Factory interface. At the top, the 'Retail\_pipeline' is selected. The main canvas shows a pipeline with three 'Copy data' activities: 'orders', 'inventory', and 'returns', connected in sequence. The 'Output' tab is active, showing a table of pipeline run details.

| Activity name | Activity status | Run start             | Duration | Input | Output |
|---------------|-----------------|-----------------------|----------|-------|--------|
| returns       | Succeeded       | 10/2/2026, 8:19:22 PM | 17s      | -     | -      |
| inventory     | Succeeded       | 10/2/2026, 8:19:03 PM | 18s      | -     | -      |
| orders        | Succeeded       | 10/2/2026, 8:18:42 PM | 20s      | -     | -      |

Microsoft Fabric / Data Factory pipeline with Orders, Inventory, and Returns copy activities

### Step-by-step explanation

- Open the retail pipeline and verify the sequence of **Copy data** activities for **orders**, **inventory**, and **returns**.
- Run or monitor the pipeline and confirm that each activity completes successfully.
- The output pane in the screenshot shows successful execution for all three activities.
- This step performs the ingestion stage: source files are copied into the data-lake environment so they can be processed by the next layer.

Retail Data Engineering • Screenshot 2 of 9

## Step 3: Read the Inventory Data in the Notebook

The screenshot shows a Microsoft Fabric notebook interface. The left sidebar contains the 'Explorer' pane with 'Data items' selected, showing a hierarchy: 'Retail\_LH' > 'Tables' > 'Bronze' > 'Inventory\_data'. The main area displays two code cells. The first cell contains a welcome message and a PySpark command to read a Parquet file from the Bronze layer: `df_invent = spark.read.parquet("abfss://Retail_Project2@oneLake.dfs.fabric.microsoft.com/Retail_LH.Lakehouse/Files/Bronze/Inventory_data.parquet")`. The second cell contains the command `display(df_invent)`. Below the code, a 'Table view' of the data is shown, displaying 12 rows of inventory data with columns: product\_id, productName, stock, last\_stocked, warehouse, cost\_price, and available.

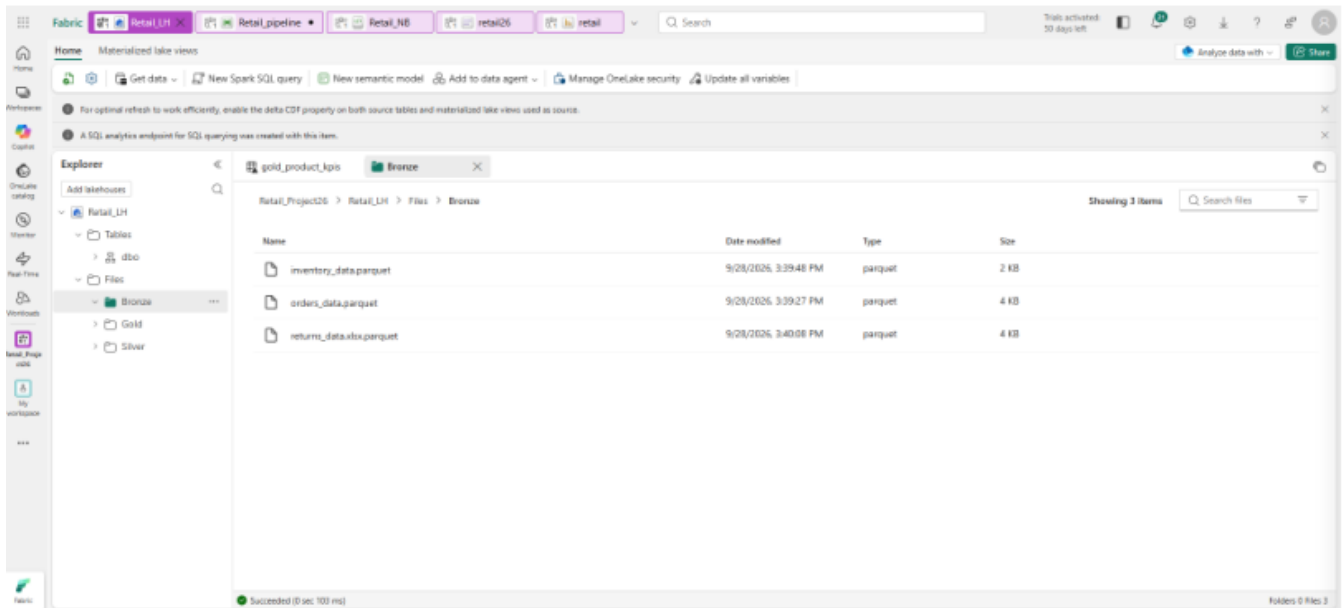
| product_id | productName         | stock       | last_stocked | warehouse  | cost_price | available |
|------------|---------------------|-------------|--------------|------------|------------|-----------|
| P001       | Apple iPhone 13     | 23 units    | 2023-06-01   | Delhi_1    | \$700      | YES       |
| P002       | Samsung Galaxy S... | 12          | 01-07-2023   | BUM2       | INR 72000  | yes       |
| P003       | Oneplus Nord        | twenty      | 2023-06-30   | mumbai@3   | Rs. 25000  | Y         |
| P004       | Nokia 3310          |             | 2023-07-05   |            | NULL       |           |
| P005       | Xiaomi note 12 pro  | 55          | 07-12-2023   | hydzone2   | \$299      | True      |
| P006       | Vivo V25            | NULL        | 2023-07-30   | chm        | 350,50     | NO        |
| P007       | Motorola Edge-40    | 0           | 25-07-2023   | Delhi_1    | USD 500    | false     |
| P008       | LG Velvet           | thirteen    | 2023-06-18   | puneRcty   | ₹32000     | Yes       |
| P009       | Realme X50          | 30          | 06-07-2023   | gurgaon-56 | USD 399    | YES       |
| P010       | Asus ROG Phone      | twenty five | 2023-07-10   | bangalore1 | Rs.58999   | Y         |
| P011       | Google Pixel 6a     | 35          | 07/15/2023   | Hyderabad5 | \$449      | yes       |
| P012       | Sony Xperia 5       | 27          | 15-07-2023   | zombabaw@  | Rs 61000   | Yes       |

Notebook view showing inventory data loaded and displayed

### Step-by-step explanation

- Open the project notebook in the analytics workspace.
- Load the inventory dataset from the Bronze location using the notebook's Spark/PySpark processing environment.
- Display the resulting DataFrame and inspect the available inventory fields.
- The screenshot shows fields including **product\_id**, **productName**, **stock**, **last\_stocked**, **warehouse**, **cost\_price**, and **available**.
- This inspection confirms that the ingested inventory data is accessible for transformation.

## Step 4: Verify the Bronze Files



Bronze layer containing the ingested parquet datasets

### Step-by-step explanation

- Navigate to the **Bronze** folder in the lakehouse / data-lake explorer.
- Verify that the ingested datasets are present as parquet files.
- The screenshot shows **inventory\_data.parquet**, **orders\_data.parquet**, and **returns\_data.xlsx.parquet**.
- The Bronze layer therefore provides the raw-ingested foundation for the cleaning and transformation stage.

## Step 5: Inspect the Bronze Inventory Dataset

The screenshot shows the Microsoft Fabric interface. The top navigation bar includes 'Fabric', 'Retail\_LH', 'Retail\_pipeline', 'Retail\_NL', 'Retail25', and 'Retail'. The main area displays a table preview of the 'inventory\_data.parquet' file. The table has 8 columns: 'id', 'product\_id', 'product\_name', 'stock', 'last\_stocked', 'warehouse', 'cost\_price', and 'available'. The data is organized into 15 rows, each representing a different product and its inventory status.

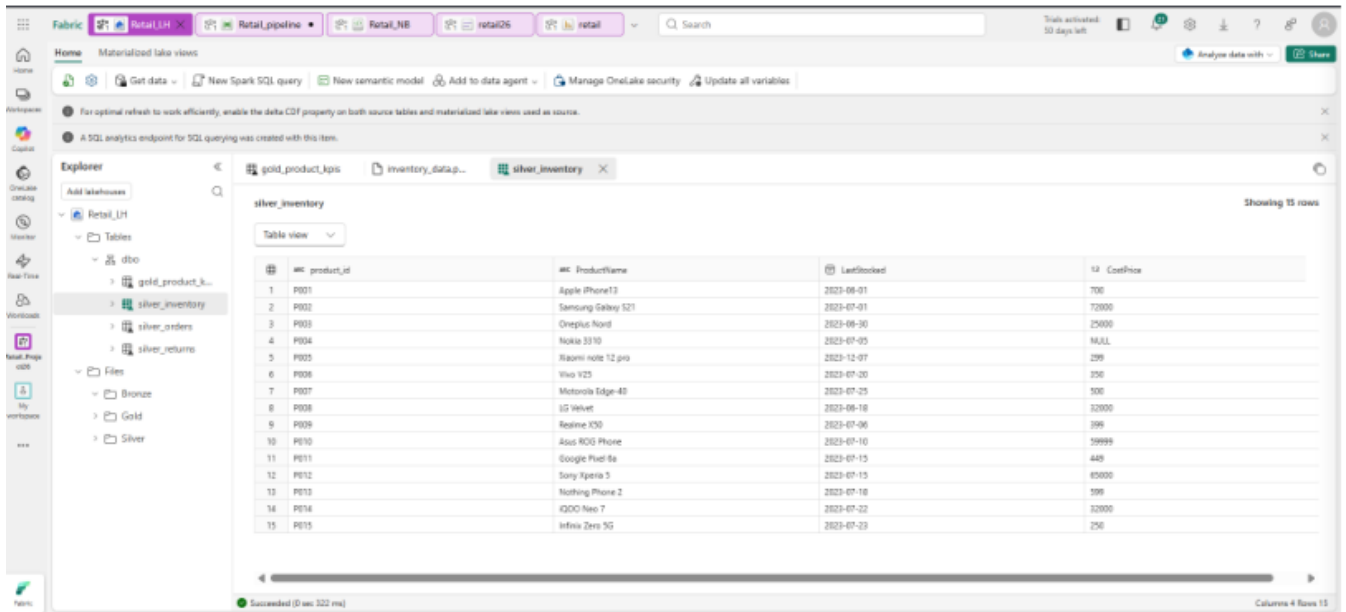
|    | id   | product_id         | product_name | stock      | last_stocked | warehouse | cost_price | available |
|----|------|--------------------|--------------|------------|--------------|-----------|------------|-----------|
| 1  | R001 | Apple iPhone13     | 25 units     | 2023/04/01 | Dalh_1       | \$700     | YES        |           |
| 2  | R002 | Samsung Galaxy S21 | 12           | 01-07-2023 | B/R02        | Rs. 13000 | yes        |           |
| 3  | R003 | OnePlus Nord       | twenty       | 2023/06/30 | mumbai@3     | Rs. 25000 | Y          |           |
| 4  | R004 | Nokia 3310         |              | 2023/07/05 |              | NULL      |            |           |
| 5  | R005 | Xiaomi note 12 pro | 55           | 07-12-2023 | HydRana2     | \$299     | True       |           |
| 6  | R006 | Vivo V25           | NULL         | 2023-07-20 | chi          | 350/50    | NO         |           |
| 7  | R007 | Motorsola Edge-40  | 0            | 25/07/2023 | Dalh_1       | USD100    | false      |           |
| 8  | R008 | Li Velvet          | fifteen      | 2023-06-18 | puneR07y     | ₹31000    | Yes        |           |
| 9  | R009 | Realme X50         | 30           | 06-07-2023 | gurgaon-56   | USD 189   | YES        |           |
| 10 | R010 | Axus ROG Phone     | twenty five  | 2023/07/10 | bangalore1   | Rs.38999  | Y          |           |
| 11 | R011 | Google Pixel 6a    | 35           | 07/15/2023 | Hyderabad5   | \$440     | yes        |           |
| 12 | R012 | Sony Xperia 5      | 22           | 15-07-2023 | coimbatore@  | Rs. 65000 | Yes        |           |
| 13 | R013 | Nothing Phone 2    | null         | 2023/07/18 | delh15       | \$590     | True       |           |
| 14 | R014 | iQOO Neo 7         | eighteen     | 2023-07-22 | chd1         | Rs.32000  | Yes        |           |
| 15 | R015 | Infinix Zero 50    | 10           | 23/07/2023 | kolkata@     | \$250     | yes        |           |

Bronze inventory parquet file opened in table preview

### Step-by-step explanation

- Open **inventory\_data.parquet** from the Bronze files.
- Use the table preview to inspect the raw inventory records before transformation.
- Review the values in fields such as product ID, product name, stock, last-stocked date, warehouse, cost price, and availability.
- This step validates the actual Bronze content and provides the source structure used for Silver-layer cleaning.

## Step 6: Validate the Cleaned Silver Inventory



The screenshot shows the Microsoft Fabric interface with the 'silver\_inventory' table selected. The table contains 15 rows of data, including product IDs, product names, last stocked dates, and cost prices. The interface also shows a sidebar with 'Explorer' and 'Data Explorer' views, and a top navigation bar with tabs for 'Retail\_LH', 'Retail\_pipeline', 'Retail\_NE', 'Retail2S', and 'Retail'.

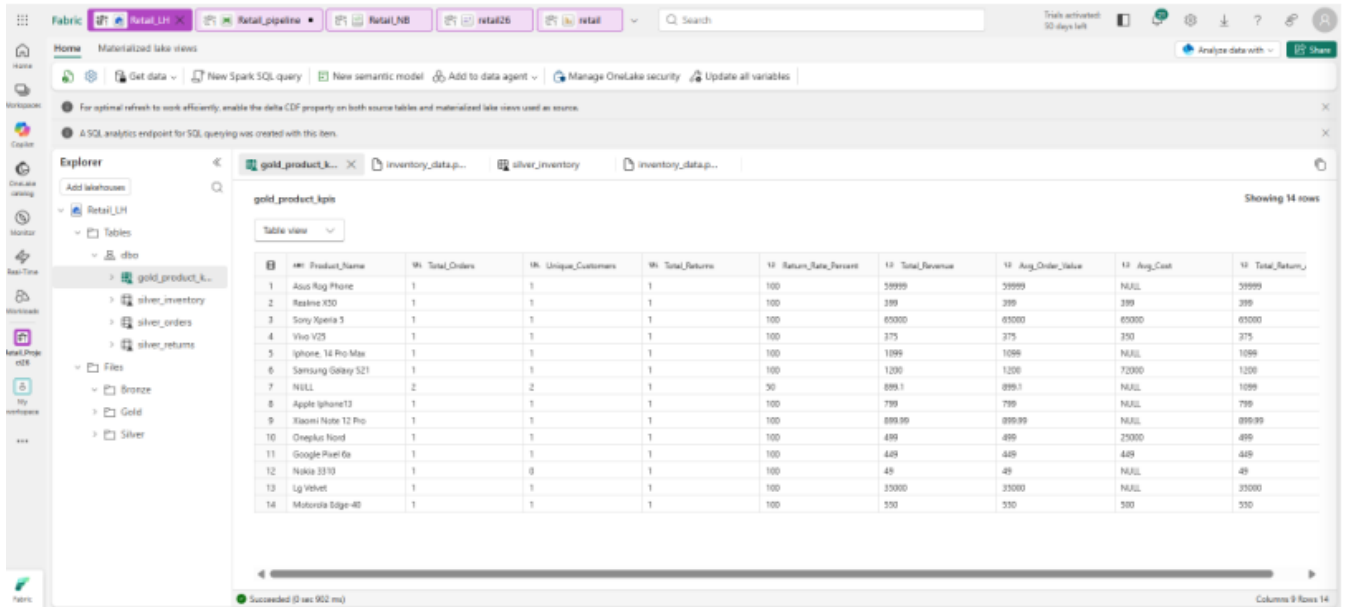
| product_id | ProductName        | LastStocked | CostPrice |
|------------|--------------------|-------------|-----------|
| P001       | Apple iPhone13     | 2023-06-01  | 700       |
| P002       | Samsung Galaxy S21 | 2023-07-01  | 72000     |
| P003       | Oneplus Nord       | 2023-06-30  | 25000     |
| P004       | Nokia 3310         | 2023-07-05  | NAUL      |
| P005       | Realme note 12 pro | 2023-12-07  | 299       |
| P006       | Vivo V25           | 2023-07-20  | 350       |
| P007       | Motorola Edge 40   | 2023-07-25  | 500       |
| P008       | LG Velvet          | 2023-06-18  | 32000     |
| P009       | Realme X50         | 2023-07-06  | 399       |
| P010       | Asus ROG Phone     | 2023-07-10  | 59999     |
| P011       | Google Pixel 6a    | 2023-07-15  | 449       |
| P012       | Sony Xperia 5      | 2023-07-15  | 65000     |
| P013       | Nothing Phone 2    | 2023-07-18  | 599       |
| P014       | iQOO Neo7          | 2023-07-22  | 62000     |
| P015       | Infima Zero 5G     | 2023-07-23  | 250       |

Silver inventory table showing the processed inventory dataset

### Step-by-step explanation

- Navigate to the **Silver** layer and open the processed inventory table.
- Verify that the inventory data has been transformed into a structured Silver dataset.
- The screenshot shows columns including **product\_id**, **ProductName**, **LastStocked**, and **CostPrice**.
- The project document specifies that the Silver layer is used to **clean** the ingested data before creating the Gold layer.

## Step 7: Build and Inspect Gold Product KPIs



Showing 14 rows

| ID | Product_Name       | Total_Orders | Unique_Customers | Total_Returns | Return_Rate_Percent | Total_Revenue | Avg_Order_Value | Avg_Cost | Total_Return_Amount |
|----|--------------------|--------------|------------------|---------------|---------------------|---------------|-----------------|----------|---------------------|
| 1  | Axol Rug Phone     | 1            | 1                | 1             | 100                 | 59999         | 59999           | NULL     | 59999               |
| 2  | Rakine X30         | 1            | 1                | 1             | 100                 | 389           | 389             | 389      | 389                 |
| 3  | Sony Xperia 1      | 1            | 1                | 1             | 100                 | 65000         | 65000           | 65000    | 65000               |
| 4  | Vivo V25           | 1            | 1                | 1             | 100                 | 375           | 375             | 375      | 375                 |
| 5  | Iphone 14 Pro Max  | 1            | 1                | 1             | 100                 | 1099          | 1099            | 1099     | 1099                |
| 6  | Samsung Galaxy S21 | 1            | 1                | 1             | 100                 | 1200          | 1200            | 72000    | 1200                |
| 7  | NULL               | 2            | 2                | 1             | 50                  | 899.1         | 899.1           | NULL     | 1099                |
| 8  | Apple Iphone13     | 1            | 1                | 1             | 100                 | 799           | 799             | NULL     | 799                 |
| 9  | Xiaomi Note 12 Pro | 1            | 1                | 1             | 100                 | 899.99        | 899.99          | NULL     | 899.99              |
| 10 | Oneplus Nord       | 1            | 1                | 1             | 100                 | 499           | 499             | 25000    | 499                 |
| 11 | Google Pixel 6a    | 1            | 1                | 1             | 100                 | 449           | 449             | 449      | 449                 |
| 12 | Nokia 3310         | 1            | 0                | 1             | 100                 | 49            | 49              | NULL     | 49                  |
| 13 | Lg Velvet          | 1            | 1                | 1             | 100                 | 35000         | 35000           | NULL     | 35000               |
| 14 | Motorola Edge-40   | 1            | 1                | 1             | 100                 | 550           | 550             | 550      | 550                 |

Succeeded (3 sec 902 ms)

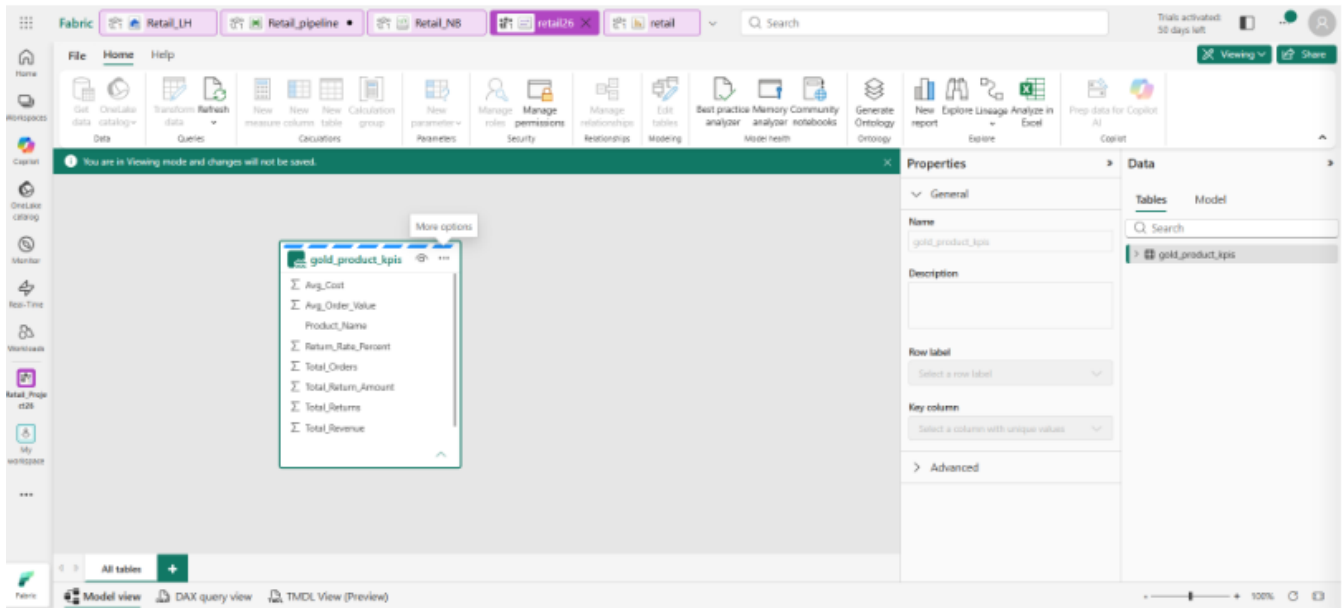
Columns: 9 Rows: 14

Gold product KPI table prepared for analytical reporting

### Step-by-step explanation

- Open the curated **gold\_product\_kpis** dataset in the Gold layer.
- Verify that the dataset contains product-level analytical measures.
- The screenshot shows fields including **Product\_Name**, **Total\_Orders**, **Unique\_Customers**, **Total\_Returns**, **Return\_Rate\_Percent**, **Total\_Return\_Amount**, **Total\_Revenue**, **Avg\_Order\_Value**, and **Avg\_Cost**.
- This Gold dataset provides the business-ready metrics that can be consumed by Power BI.

## Step 8: Connect the Gold Dataset to the Power BI Model

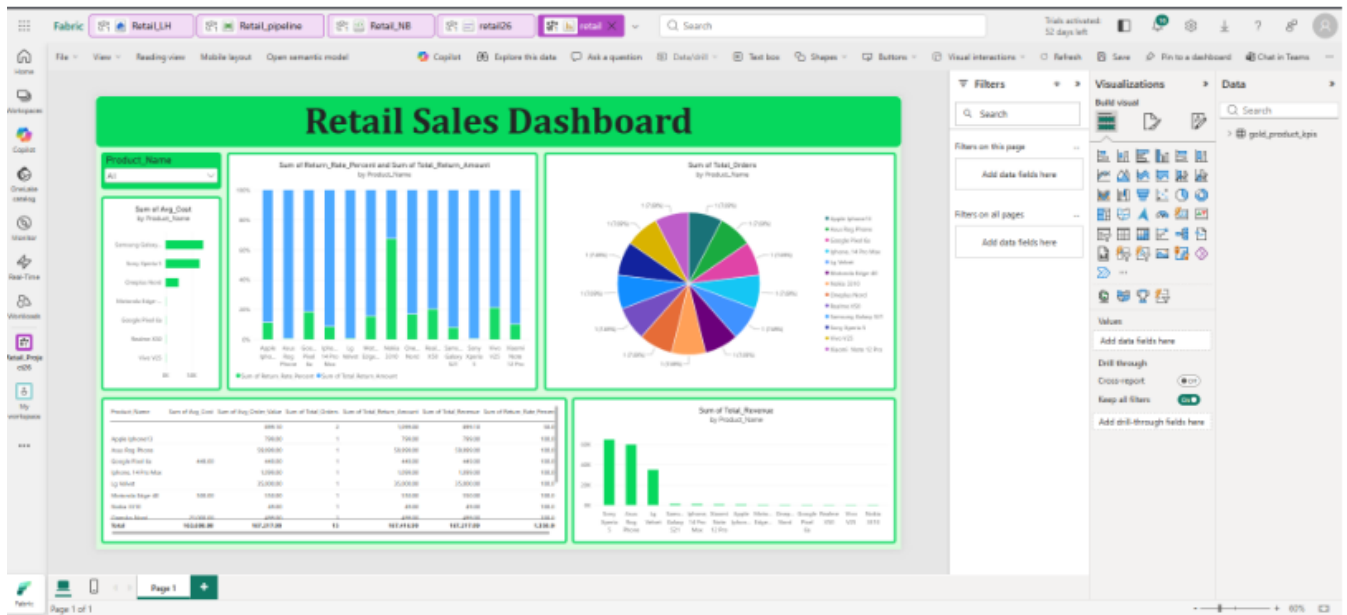


Power BI model view using the gold\_product\_kpis dataset

### Step-by-step explanation

- Open Power BI and connect the curated Gold dataset to the semantic model.
- The model view shown in the screenshot contains the **gold\_product\_kpis** table.
- The table exposes the product-level KPI fields needed for reporting and visualization.
- This creates the bridge between the curated Gold layer and the final business dashboard.

## Step 9: Build the Retail Sales Dashboard



Power BI Retail Sales Dashboard built from the curated Gold data

### Step-by-step explanation

- Open the Power BI report after the Gold dataset has been connected to the model.
- The dashboard is titled **Retail Sales Dashboard** and contains multiple visuals for product-level analysis.
- The page includes a product filter, charts, a product KPI table, a revenue visualization, and a return-related visualization.
- Use the report to explore the curated retail metrics delivered from the Gold layer.
- This completes the documented flow from source ingestion through ADLS, processing, Silver cleaning, Gold KPI preparation, and Power BI reporting.